Breast Cancer Prediction using Machine Learning

Week 6 Final Project Report

Artificial Intelligence Internship

Submitted by: Rohit Kumar Gupta

1. Introduction

Breast cancer is one of the most common cancers, and finding it early greatly improves the chance of successful treatment. When a lump is found, a small sample of cells is examined and measurements such as radius, texture and area of the cell nucleus are recorded. A doctor then decides whether the tumour is **malignant** (cancerous) or **benign** (not cancerous).

In this project I used Python, Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn to build a machine learning model that predicts the diagnosis from these measurements. The project follows the complete workflow: problem definition, data collection, data cleaning, data analysis, model development, training, prediction, evaluation and conclusion.

2. Problem Statement

Problem: Given 30 measurements of a tumour's cell nucleus, predict whether the tumour is malignant or benign.

Objective: Clean and understand the data, find the features most related to cancer, train a classification model, and evaluate it using accuracy, precision, recall and a confusion matrix.

Why ML is suitable: It is a supervised binary classification problem with labelled data. A person cannot easily study 30 numbers together for every patient, but a model can learn the pattern from past cases.

Real-world application: A second-opinion tool that supports doctors in early screening. It supports the doctor and does not replace the doctor.

3. Dataset

The **Breast Cancer Wisconsin (Diagnostic)** dataset was used. It is available in the scikit-learn library and in the UCI Machine Learning Repository. I loaded it with scikit-learn and saved it as *data/breast_cancer.csv*.

Property	Value
Number of records	569
Number of features	30 (all numeric)
Target variable	target (0 = malignant, 1 = benign)
Malignant cases	212 (37.3%)
Benign cases	357 (62.7%)

The 30 features are the **mean**, **standard error** and **worst** (largest) value of 10 measurements: radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension.

Figure 1: First 5 records of the dataset

Figure 2: Last 5 records of the dataset

4. Data Preprocessing

Using Pandas and NumPy, I performed these steps: loaded the CSV, displayed the first and last records, checked rows and columns (569 x 31), checked data types (all float64 except the integer target), looked for missing values and duplicates, selected the features and target, and applied feature scaling.

Figure 3: Dataset overview

Missing values and duplicates: The dataset had 0 missing values and 0 duplicate rows. I still wrote the handling code (fill missing numeric values with the median, and drop_duplicates) so the notebook would work even if the data changed. Because the dataset was already clean, the before and after numbers are the same.

Figure 4: Before and after cleaning

Features and target: X = the 30 numeric features, y = target. No encoding was needed because all features are numeric.

Feature scaling: The features have very different ranges (mean area is in hundreds while mean smoothness is about 0.1), so StandardScaler was applied. It was fitted only on the training data to avoid data leakage.

5. Exploratory Data Analysis

Figure 5: Number of patients by diagnosis

Insight: 357 patients (62.7%) are benign and 212 (37.3%) are malignant. The classes are slightly imbalanced, so accuracy alone is not enough and precision and recall are also checked.

Figure 6: Distribution of mean radius

Insight: Malignant tumours are larger. Average mean radius is about 17.5 for malignant and 12.1 for benign tumours, and the two histograms overlap only a little.



Figure 7: Mean radius vs mean area

Insight: Radius and area increase together. Malignant points are in the upper right and benign points in the lower left, so the classes are well separated, with only a few overlapping points in the middle.

Figure 8: Mean concavity by diagnosis

Insight: Malignant tumours have much higher concavity (average 0.161) than benign ones (0.046), more than three times higher, and they vary more from patient to patient.



Figure 9: Correlation heatmap of the mean features

Insight: Radius, perimeter and area are almost perfectly correlated (about 0.99 to 1.0), so they carry nearly the same information. Concave points, concavity and compactness are also strongly related. The target column shows strong negative correlation with concave points, perimeter and radius (negative because benign = 1, so bigger values mean a higher chance of malignant).

Figure 10: Top 10 features by absolute correlation with the target

Insight: The strongest features are worst concave points (0.79), worst perimeter (0.78), mean concave points (0.78) and worst radius (0.78). Shape and size measurements are the best indicators, while fractal dimension and texture error are almost useless on their own.

6. Model Development

The data was split into **80% training (455 records)** and **20% testing (114 records)** with stratify=y so both sets keep the same malignant/benign ratio and random_state=42 for repeatable results.

I selected **Logistic Regression** for these reasons:

1. It is designed for binary classification and gives a probability for each patient.
2. Many features have an almost linear relationship with the target (seen in the heatmap and scatter plot).
3. It is simple, fast and easy to explain, which matters in medical problems.
4. Its coefficients show which features push the prediction towards malignant or benign.

To check my choice I also trained a Decision Tree, KNN and a Random Forest and compared them on the same test set.

7. Training and Prediction

The model was trained with *model.fit(X_train_scaled, y_train)* and predictions were made with *model.predict(X_test_scaled)* on the 114 unseen test records. The model also gives the probability of the tumour being benign using predict_proba.

Figure 11: Comparison of the four models

Figure 12: Accuracy of different models

Logistic Regression gave the best accuracy (98.2%) so it was chosen as the final model.

8. Model Evaluation

Metric	Score
Accuracy	98.25%
Precision	0.9861
Recall	0.9861
F1-score	0.9861
5-fold cross-validation (average accuracy)	98.07%

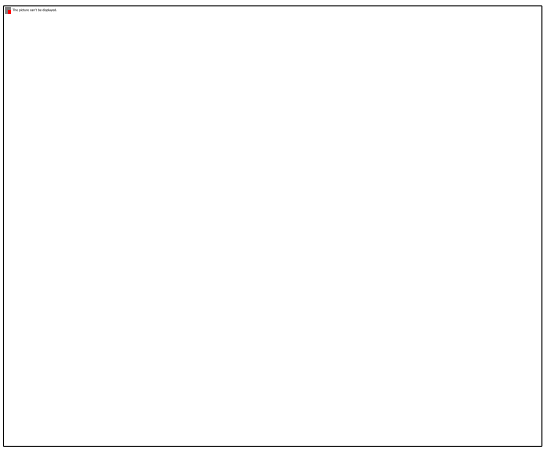


Figure 13: Confusion matrix

Out of 114 test patients, the model predicted 41 malignant cases correctly and 71 benign cases correctly. There was only **1 malignant case predicted as benign** (the more dangerous mistake) and **1 benign case predicted as malignant** (a false alarm).

The cross-validation scores (98.2%, 98.2%, 97.4%, 97.4%, 99.1%) are close to the test accuracy, so the model is stable and not just lucky on one split.

Figure 14: Top 10 features used by the model

9. Conclusion

In this project I completed the full machine learning workflow on the Breast Cancer Wisconsin dataset. I loaded and cleaned the data (569 records, 30 features, no missing values or duplicates), explored it with six visualizations, and found that malignant tumours are larger and have more concave edges than benign ones.

A Logistic Regression model trained on 80% of the data reached **98.2% accuracy** on the unseen test data, with precision and recall of about 98.6%, and it performed better than Decision Tree, KNN and Random Forest. This meets my expected outcome of more than 95% accuracy.

Limitations: The test set has only 114 patients, so a single mistake changes the accuracy by almost 1%. The dataset is small and comes from one source. This model is for learning and is not approved for real medical use.

10. Future Scope

1. Tune hyperparameters with GridSearchCV.
2. Build an Artificial Neural Network with TensorFlow / Keras and compare it with this model.
3. Use a larger dataset collected from many hospitals.
4. Focus more on recall so that no cancer case is missed.
5. Make a small web app (Streamlit or Flask) where a doctor can enter the measurements and get a prediction.
6. Try feature selection to remove highly repeated features like radius, perimeter and area.

References

1. Breast Cancer Wisconsin (Diagnostic) Data Set, UCI Machine Learning Repository.
2. Scikit-learn documentation: scikit-learn.org
3. Pandas, NumPy, Matplotlib and Seaborn documentation.